

黑板

P1157

$n=5, r=3$

C_5^3

全集 $\{1, 2, 3, 4, 5\}$

子集 $\{1, 2, 3\} \{1, 2, 4\} \{1, 2, 5\}$

$\{1, 3, 4\} \{1, 3, 5\}$

$\{1, 4, 5\}$

$\{2, 3, 4\} \{2, 3, 5\}$

$\{2, 4, 5\}$

$\{3, 4, 5\}$

输出一个子集枚举

给定一个集合，枚举所有可能的子集

($n=5$ 中取出 $r=3$)

三种子集枚举

1. 增量构造法

一次取出一个元素放到集合A中。(A是构造)

定序：规定集合中所有元素从小到大排序，

A
集合

防止在输出打印时，出现

$\{1, 2\}, \{2, 1\}$ 二次

递归
函数

```
void print_subset(int n, int *A, int cur)
{
```

```
    for(int i=0; i<cur; i++)
```

```
        printf("%d ", A[i]);
```

```
    printf("\n");
```

```
    int S = cur ? A[cur-1]+1 : 0;
```

{ 递归结束
打印输出

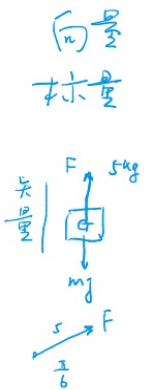
```

int S = cur ? A[cur-1] + 1 : 0;
for(int i=S; i<n; i++)
{
    A[cur] = i;
    print_subset(n, A, cur+1);
}
}
构造子集A

```

2. 位向量法

构造一个位向量 $B[i]$, 不是直接构造 A 本身
 $B[i] = 1$, iff i 在子集 A 中.



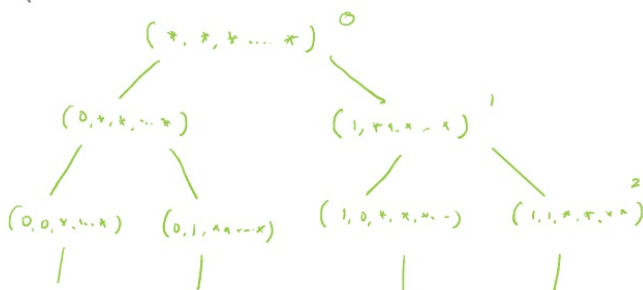
```

Void print_subset(int n, int *B, int cur)
{
    if (cur == n)
    {
        for(int i=0; i<cur; i++)
            if (B[i]) printf("%d ", i);
        printf("\n");
        return;
    }
    B[cur] = 1;
    print_subset(n, B, cur+1); // 选这个元素
    B[cur] = 0; // 不选
    print_subset(n, B, cur+1);
}

```

位向量法

解答树



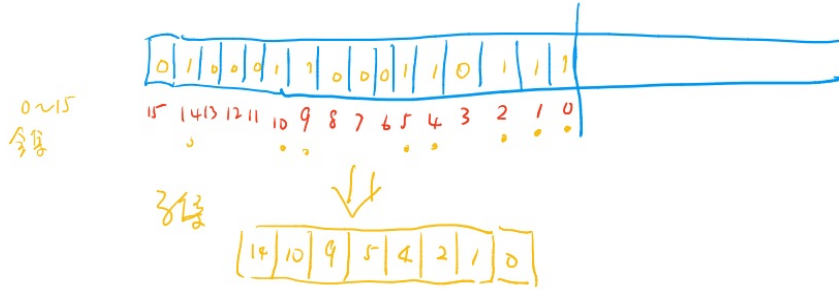
这是一个 $n+1$ 层的二叉树
 (cur 范围从 $0 \sim n$ 编号)
 第0层有一个结点
 第1层有两个结点
 第2层有四个结点
 第3层有八个结点
 第 i 层有 2^i 个结点

总数为 $1 + 2 + 4 + 8 + \dots + 2^n = 2^{n+1} - 1$

3. 二进制法

3. 二进制法

用二进制来表示全集 $\{0, 1, 2, \dots, n-1\}$ 的子集 S
 从右到左排列 (首位是 0 开始编号), 元素 i 是否
 在 S 中, 0 表示不在, 1 表示在



位运算

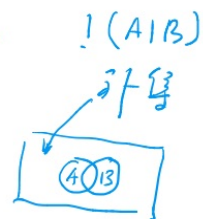
& 与
 | 或
 ! 非
 ^ 异或 (开关性)



	A	B	A & B	A B	A ^ B
二进制	1 0 1 1 0	0 1 1 0 0	0 0 1 0 0	1 1 1 1 0	1 1 0 1 0
集合	{1, 2, 4}	{2, 3}	{2}	{1, 2, 3, 4}	{1, 3, 4}

集合
 操作
 并集
 交集
 差集
 补集

空集为 0
 全集 $\{0, 1, 2, 3, \dots, n-1\}$
 二进制为 $n-1$,
 也就是 $2^n - 1$



编译器 GCC

在 GCC 中, 有一个内置函数

`_builtin_popcount()`

`_builtin_popcount(13) ==> 3`
 ↑
 二进制

`void print-subset(int n, int s)`

void print-subset (int n, int s)

二进制的枚举

```

{
    for (int i=0; i<n; i++)
        if (S & (1<<i)) printf("%d ", i);
        printf("\n");
}

```

枚举子集

```

for (int i=0; i<=(1<<n); i++)
    print-subset(n, i);

```

枚举子集用 = 二进制 代码量最小, 最简单

最优配对问题

因为每个点都要配对, 因此就转换成多阶段决策.

1. 确定 P_0 的配对, 然后再是 $P_1, P_2 \dots P_{n-1}$

2. 设 $d(i)$, 表示前 i 个点的两两配对的最小距离和

3. 考虑第 i 个点的 n 情况: (与谁配对)

假设: 第 i 个点与点 j 配对 ($j < i$)

那么, "把前 $i-1$ 个点中, 除了 j 之外的其他点两两配对", 所以 $d(i)$ 不能表示全部情况, 需要增加一项

设 $d(i, S)$, 表示前 i 个点中, 位于集合 S 中的元素两两配对的最小距离和.

$$d(i, S) = \min \{ |P_i P_j| + d(i-1, S - \{i\} - \{j\}) \mid j < S \}$$

$|P_i P_j|$ 表示 P_i 和 P_j 之间的距离

S 全集表示:

用整数表示集合 $\{0, 1, 2, 3, \dots, n-1\}$

任意子集 (subset)

子集枚举, 子集的二进制表示

距离和

```

for (int i=0; i<n; i++)

```

7

```
for(int i = 0; i < n; i++)
```

```
for(int S=0; S<(1<n); S++)
```

$$d[i][s] = INF;$$

```
for(int j=0; j<i; j++)
```

$$if(S \wedge (1 \leq j))$$
$$d[i][5] =$$
$$\min(d[i][s], \text{dist}(i, j)) +$$
$$d[i-1][S \oplus (1 \ll i) \wedge (1 \ll j)]$$

1